yast-on-arch — User Guide

openSUSE's YaST system-configuration tool, built and running on Arch Linux. This guide describes version **0.3.1** (`~/libyui-port/yast --version`; changes are listed in CHANGELOG.md).

This guide is for anyone installing yast-on-arch, from people who remember YaST from SuSE and just want it back, to people who want to port more modules. It starts in plain words, then gives the detail, then every option. A glossary is at the end.

> **Read section 7 (Safety) before you open any module as root.** YaST configures your real system. There is no "preview" mode, and several modules write files as soon as you press OK.

---

1. In plain words

YaST ("Yet another Setup Tool") is the all-in-one control panel of SUSE and openSUSE Linux. One window, a list of groups (Hardware, System, Network, Security…), and inside each group a module that edits one part of the system: users, services, boot loader, firewall, time zone, Samba, NFS, and so on. It has three faces: a **Qt** window, a **GTK** window, and the classic blue **text mode** (ncurses) that works over SSH.

YaST was only ever packaged for SUSE. openSUSE Leap 16 dropped the graphical YaST, and the last port to another distribution went quiet around 2010.

yast-on-arch makes it build and run on Arch Linux. It doesn't contain YaST's code. It is a script (`bootstrap.sh`) plus a set of small patches. The script downloads the official YaST and libyui sources, applies the Arch fixes, and installs everything into one folder in your home: `~/libyui-port/`. Nothing is copied into `/usr`.

Who it is for

- People who liked YaST on SUSE and want it on Arch.
- People who prefer one consistent tool for users, services, NFS/Samba, firewall and time settings, in a GUI or over SSH in text mode.
- Developers interested in porting YaST modules, or in the Ruby 3.4 fix (section 13).

What it can't do

- **Install or remove software.** YaST's package manager is built on openSUSE's libzypp/RPM. The Software group opens **Octopi** (a pacman/AUR GUI) instead, if it is installed, and adds a pacman **Package Cache Cleaner**.
- **Install Linux.** The YaST installer parts are not ported.
- Everything that depends on SUSE-only services (SuSEfirewall, wicked, AutoYaST profiles, SUSE registration) is missing or partial.

---

2. Requirements

need	detail
Distribution	**Arch Linux** (or Arch-based). The script warns and continues on other systems, but package names are Arch's.
Account	A normal user with `sudo` rights. YaST modules need real root (uid 0); being in `wheel` alone isn't enough.
Desktop	Any, for text mode. For the Qt/GTK windows as root on Wayland, **XWayland** must be available (the launcher uses it; `xorg-xhost` is installed by the build). KDE Plasma gets matching dark/light themes automatically. For starting root modules from the menu or the Control Center without passwordless `sudo`: `polkit` with a polkit agent (every major desktop runs one).
Disk / time	A few GB in `~/libyui-port` for sources and build trees; the first build takes a while (it compiles libyui, yast-core, libstorage-ng and more).
Network	Only during the build (git clones of upstream). Nothing is downloaded at runtime.
Optional	`octopi` (Software group), `pacman-contrib` (`paccache`, for the cache cleaner), `konsole` (text-mode menu entry), and the services each module manages (`samba`, `nfs-utils`, `firewalld`, `chrony`, `ypbind-mt`, `tftp-hpa` ...).

What `bootstrap.sh` installs on your system

It runs `sudo pacman -S --needed --noconfirm` for these packages (already-installed ones are skipped):

```
cmake ninja boost qt5-base qt5-x11extras gtk3 ncurses gettext libxcrypt pkgconf git
ruby ruby-rake bison flex libtool automake autoconf dejagnu
docbook-xsl libxslt perl-xml-writer fdupes ruby-nokogiri ruby-augeas augeas
libxml2 json-c swig perl-crypt-smbhash xorg-xhost doxygen
```

It also installs these Ruby gems **for your user only** (`gem install --user-install`): `fast_gettext` (pinned below 3.0, because Arch's 3.1 is too new), `cheetah`, `simpleidn`, `abstract_method`, `yast-rake`, `prime`, `cfa`, `cfa_grub2`, `ruby-dbus`.

Print the list without installing anything:

```
./bootstrap.sh --deps
```

Outside `~/libyui-port` it also writes four menu entries to `~/.local/share/applications` (section 3; skip with `YAST_NO_MENU_ENTRIES=1`) and a systemd drop-in for NIS, the latter only if `ypbind-mt` is installed (section 8). Programs you add to the Control Center yourself (section 5) are kept in `~/.local/share/yast-on-arch/`.

---

3. Install

```
git clone https://github.com/eugenesflanagin-spec/yast-on-arch.git
cd yast-on-arch
./bootstrap.sh
```

Run it as your normal user (not with `sudo`). It asks for your password once, for `pacman`.

To build somewhere other than `~/libyui-port`:

```
YAST_ARCH_ROOT=/data/yast ./bootstrap.sh
```

What the build does, in order

1. Installs the packages and gems above, then clones each upstream repository at the commit the patches were made for (section 9).
2. Creates a `lib64 → lib` link inside the prefix (YaST's build files assume `lib64`).
3. Builds an **ncursesw header shim** (Arch keeps the wide-character curses headers directly in `/usr/include`, while SUSE/Debian use `/usr/include/ncursesw/`).
4. Builds **libyui** core, ncurses, Qt and GTK front-ends.
5. Builds **yast-devtools** (patches 01–02), **yast-core**, **yast-ycp-ui-bindings**, **yast-ruby-bindings** (patch 03, the Ruby 3.4 fix) and **yast-perl-bindings**.
6. Installs the YaST modules (pure Ruby, copied with `rake install`), applying the Arch patches per module (section 12).
7. Builds **libstorage-ng** (patch 08) and installs **yast-storage-ng**.
8. Installs the Arch stand-ins (`Pkg`, `InstURL`, `Packages`, `SLPAPI`, `PackageSlideShow`), Arch copies of the `cfa/grub2` helpers, the Arch-only modules, an augeas lens and the `.probe` hardware agent.
9. Builds the classic **Qt Control Center** (patches 07, 20, 21).
10. Installs the launchers `~/libyui-port/yast`, `~/libyui-port/yui-demo`, `~/libyui-port/yast-env.sh` and `~/libyui-port/yast-apps` (behind `yast add-app`), and the menu entries.
11. Puts back any programs and sections you added yourself with `yast add-app` (section 5).

It prints `patch <name>: applied` (or `already applied`) for every patch, `ok <module>` or `FAIL <module>` for each YaST module, then `Done.` and a short usage summary. If a patch does not apply, the build **stops** with an error naming it (section 9).

One-time step after the build

```
sudo mkdir -p /var/log/YaST2           # Arch doesn't have it; root YaST logs there
```

Menu entries

The build installs four menu entries into `~/.local/share/applications`, with the real path of your launcher filled in: **YaST Control Center**, **YaST Control Center (GTK)**, **YaST Control Center (Text Mode)** (opens in your desktop's terminal) and **YaST Services Manager**. If a file with the same name already exists and wasn't written by the build, it is left alone and the build says so; delete it and re-run to get the generated one. `YAST_NO_MENU_ENTRIES=1 ./bootstrap.sh` skips this step.

---

4. Everyday use

The launcher is `~/libyui-port/yast`. Optionally add an alias: `alias yast="$HOME/libyui-port/yast"`.

```
~/libyui-port/yast                          # list every installed module (client) and
        exit
~/libyui-port/yast control-center           # the classic Qt Control Center, as your user
~/libyui-port/yast -r -q services-manager   # one module, Qt window, as root
~/libyui-port/yast -r services-manager      # one module, classic blue text mode, as root
~/libyui-port/yast -r -g menu               # GTK "menu" control center, dark, as root
~/libyui-port/yast -r menu                  # text-mode menu (works over SSH)
```

The Control Center

`yast control-center` (or `yast cc`) opens the real yast2-control-center: search box, group list on the left, module icons on the right. It runs as **your** user, so it follows your Qt/KDE theme (dark mode included). When you click a module, it starts that module through `yast -r -q <module>`, which asks `sudo` for root. Root modules get a copy of your `kdeglobals`, so they look the same.

> **How a click becomes root.** The Control Center starts the module in the background, with no terminal to type a password into. The launcher then uses `sudo -n` if your `sudo` works without a password prompt (NOPASSWD or recently used), otherwise `pkexec`, which shows your desktop's graphical password dialog (needs `polkit` and a polkit agent, standard on KDE, GNOME, Xfce...). If neither is possible it opens the command in a terminal window to ask for the `sudo` password, and as a last resort shows a notification explaining what to run. Output goes to `$XDG_RUNTIME_DIR/yast-on-arch.log`. Menu entries started without a terminal behave the same way.

Entries without a YaST module behind them (Octopi) are started as your user, detached.

Search: type in the search box and both the module grid and the group list narrow to matches.

Section colours. The *Section colours* box under the search field has three settings:

- **Off**: no colours, as YaST always looked.
- **Headers** (the default): each section header gets a colour, with a matching stripe next to the section in the group list.
- **Whole block**: as Headers, and the whole section, programs included, is tinted down to the next header.

Related sections share a colour: every "Graphics - ..." section is pink, every "Games - ..." section green, Hardware orange, Emulators magenta, and so on. Your choice is saved in `~/.config/yast-on-arch/section-colors.ini`. You can change any colour there:

```
[General]
Mode=blocks          ; off, headers or blocks

[Colors]
Graphics=#e05c8a     ; a genre (the part of a section name before " - ")
My Tools=#40a0ff     ; or one section by its full name

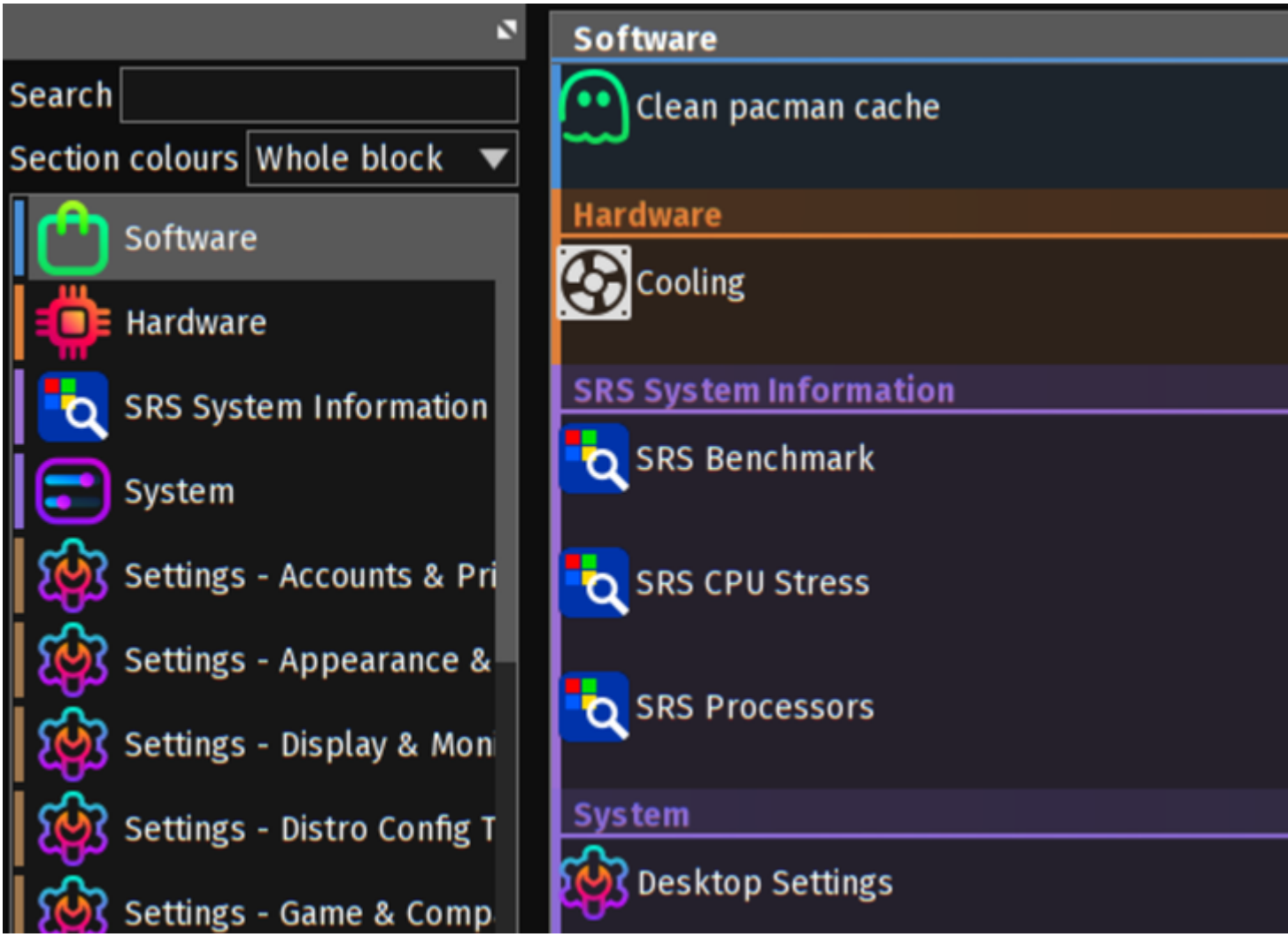
[Aliases]
My Scripts=Development                ; this section takes the Development colour
More - System - Containers=Games      ; works for any section, by its full name
```

If you have sections called "More - System - *category*" (for example ones you added yourself), each gets a colour of its own instead of all sharing one. Where a category matches a genre it uses that genre's colour: Networking -> Network, Security & Accounts -> Security, Hardware & Drivers -> Hardware, Gaming Helpers -> Games, System Info & Monitors -> Information, Other -> Utilities. The rest (AI & LLM, Containers, Disks & Partitions, Terminals & Shell, ...) have hand-picked colours that are easy to tell apart, and a category the Control Center doesn't know gets a new colour as described below.

Two ways to change a section's colour:

- `[Colors]` : give the section's full name a colour, e.g. `More - System - Containers=#2e86c1`. A full name always wins over its genre.
- `[Aliases]` : put the section into another genre, e.g. `My Scripts=Development`. It then follows that genre's colour, also when you change the genre's colour later. Your aliases are added to the built-in ones (such as Virtualization -> Emulators) and replace a built-in one with the same name. The genre name must match exactly, including capitals. If the Control Center already saved a colour for that section under `[Colors]`, delete that line first: a full name always wins.
- Colours the Control Center picks by itself for a new system category are saved as `System: <category>` (for example `System: Containers`).

Restart the Control Center after editing the file by hand.

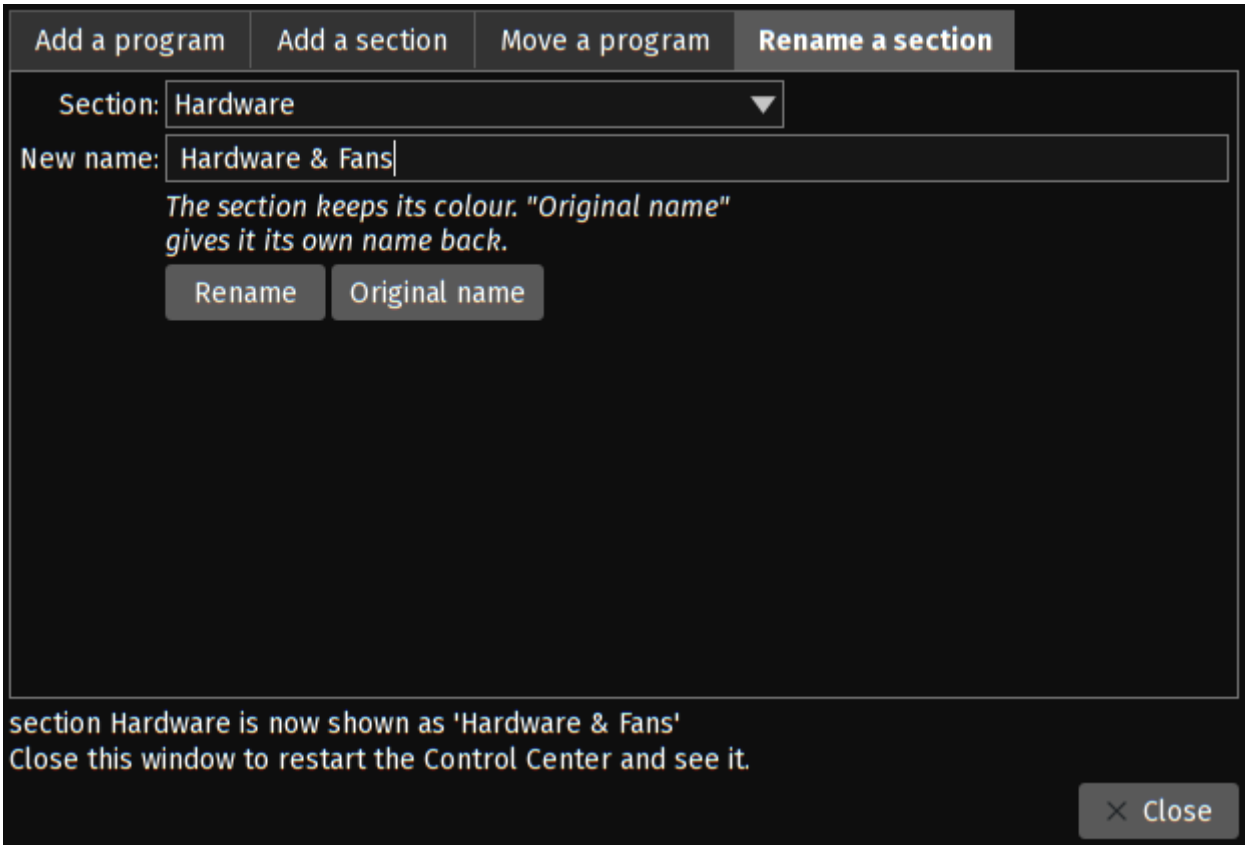


When the Control Center saves this file (you change the mode, or a new genre gets its colour) it rewrites it in Qt's format: comments are dropped and special characters in names are written as `%20` and the like. That is normal; it still reads the same.

A genre the Control Center hasn't seen before (for example one you made with *Add a section*) gets the colour furthest from every colour already in use, written to that file so it stays the same. A renamed section keeps its colour.

Add or change... (the button under *Section colours*) opens a window with four tabs:

tab	does
Add a program	name, command, icon, description, search words, section, "run in a terminal"
Add a section	name, genre and icon. Pick an existing genre to share its colour (the section is called "Genre - Name"), type a new genre for a colour of its own, or leave it empty.
Move a program	show any program, also YaST's own modules, in another section. *Put back* returns it to its own section.
Rename a section	show any section, also YaST's own, under another name. *Original name* undoes it.



You can also right-click a program and choose *Move to another section…*. When you close the window after a change, the Control Center restarts by itself to show it. The window runs the same `yast add-app`, `add-section`, `move-app` and `rename-section` commands as section 5, so the terminal and the window always agree.

The three front-ends

flag	front-end	notes
(none)	ncurses (text)	Works in any terminal and over SSH. Mouse works in most terminals. Keys: `Tab` /arrows to move, `Enter` to press, `Alt` +highlighted letter for shortcuts, `F10` = OK/Next, `F9` = Cancel, `F1` = help.
`-q`	Qt	Native look under KDE/Plasma.
`-g`	GTK	Forced to Adwaita (dark by default) for reasons explained in section 11 (Troubleshooting).

Modules

`yast` with no arguments lists every client in the prefix. The main ones (names are what you type after `yast`):

group	modules
System	`services-manager` (systemd units), `bootloader` (GRUB), `language`, `keyboard`, `timezone`, `sysconfig`, `alternatives`, `journal` (systemd journal viewer), `kde_settings` (Arch-only, see below)
Hardware	`disk` / `partitioner` / `storage` (libstorage-ng; **destructive if you apply changes**)
Network	

group	modules
	`lan`, `host` (/etc/hosts), `dns`, `routing`, `proxy`, `remote`, `vpn`, `nfs` (client), `nfs-server`, `samba-client`, `samba-server`, `nis` (NIS client), `tftp-server`, `iscsi-client`, `ntp-client`, `ldap_browser`
Security	`users`, `groups`, `firewall` (firewalld), `security` (login and kernel security settings), `apparmor`
Software	Octopi (if installed), `pacman_cache` (Arch-only, see below)

Modules with `_auto`, `_proposal`, `_finish` or `inst_` in their names are AutoYaST/installer pieces and are not meant to be run directly.

The two Arch-only modules

- **KDE Plasma Settings** (`kde_settings`, System group). Lists the KDE configuration modules (KCMs) installed on your system and opens the one you pick in `kcmshell6`, or System Settings. It runs as **your user**, on purpose: KCMs write per-user settings, and running them as root would write to the wrong home. Needs KDE Plasma (`kcmshell6`).
- **Package Cache Cleaner** (`pacman_cache`, Software group). A front-end to `paccache` for `/var/cache/pacman/pkg`. It needs root (`yast -r pacman_cache`) and `pacman-contrib`. Choices:

choice	runs
Keep the 3 newest versions of each package	`paccache -rk3`
Keep only the newest version	`paccache -rk1`
Remove cached packages that are no longer installed	`paccache -ruk0`
Remove every cached package	`paccache -rk0`

  Preview runs the same choice as a dry run (`-d`) and only lists files. **Remove Now** asks for confirmation first.

libyui demos

```
~/libyui-port/yui-demo                # list libyui's example programs
~/libyui-port/yui-demo HelloWorld     # text mode
~/libyui-port/yui-demo -q ManyWidgets # Qt
```

These only show widgets and change nothing.

---

5. Add your own app or section

In plain words: the Control Center is not only for YaST's own modules. You can put any program in it, in one of the existing sections or in a new section of your own. You don't need root for this, and your additions survive rebuilding the port.

The quick way: `yast add-app`

```
# a program, in the existing "Miscellaneous" section (ID Misc)
~/libyui-port/yast add-app --name "Disk Usage" --exec "filelight" --icon filelight

# a terminal program: --terminal opens it in a window that stays open
~/libyui-port/yast add-app --name "Process Monitor" --exec "htop" --terminal \
    --icon utilities-system-monitor --comment "See what is using the CPU" --keywords
        top,cpu

# a new section, and a second program in it
~/libyui-port/yast add-app --name "My Script" --exec "$HOME/bin/my-script.sh" \
    --new-section "My Tools" --section-icon applications-utilities
~/libyui-port/yast add-app --name "Notes" --exec "kate $HOME/notes.txt" --section MyTools
```

Then close and reopen the Control Center. The entries also show up in the text-mode and GTK menu (`yast -r menu`, `yast -r -g menu`).

command	does
`yast add-app --name NAME --exec COMMAND [options]`	add a program
`yast list-sections`	every section ID you can give to `--section` (stock and yours)
`yast list-apps`	the programs you added (ID, section, name)
`yast remove-app NAME or ID`	remove one of your programs
`yast add-section --name NAME [--genre GENRE] [--icon ICON]`	make an empty section; with `--genre` it is called "GENRE - NAME" and shares that genre's colour
`yast move-app NAME --to ID`	show any program (yours or YaST's) in another section; `--reset` puts it back
`yast rename-section ID --to NAME`	show any section (yours or YaST's) under another name; `--reset` undoes it
`yast list-layout`	your moves and renames
`yast remove-section ID`	remove one of your sections (it must be empty; programs you moved into it go back home)
`yast sync-apps`	copy your entries into YaST again (the build does this for you)
`yast add-app --help`	all options

Options of `add-app`:

option	meaning
`--name NAME`	the label under the icon (required)
`--exec COMMAND`	the command to run, as you would type it in a shell (required). It runs as **your** user.
`--icon ICON`	an icon name from your icon theme (`utilities-terminal`, `firefox`…) or the full path to an image
`--comment TEXT`	one-line description
`--keywords a,b,c`	extra words the Control Center search finds it by
`--section ID`	an existing section; default `Misc` (Miscellaneous)
`--new-section NAME`	create a section called NAME (its ID is NAME without spaces) and put the program there
`--section-icon ICON`	icon for the new section (default `folder`)
`--terminal`	wrap the command in a terminal window that stays open (`konsole`, `xfce4-terminal` or `xterm`)
`--id ID`	the file name part (default: NAME without spaces and punctuation)
`--replace`	overwrite your own entry with the same ID
`--force`	accept a command whose program is not installed yet

What it checks: the name and command are one line; the command has no `%U`/`%f`-style codes (they would be passed on literally) and no backslashes (put complicated commands in a script); the program exists; the section exists; and the finished file passes `desktop-file-validate`. It refuses to overwrite a YaST entry or anything it didn't create.

Where your entries live: `~/.local/share/yast-on-arch/applications/*.desktop`, and new sections in `…/applications/groups/`. They are copied into `~/libyui-port/prefix/destdir/usr/share/applications/YaST2/` (and its `groups/`), where YaST reads them. Each file carries the line `X-YaST-On-Arch-User=true`; only files with that line are ever replaced or deleted there, so stock modules and files you placed by hand are never touched. Back up the first folder and you have backed up your additions.

Moving programs and renaming sections

```
~/libyui-port/yast move-app "Clean pacman cache" --to Hardware   # by the name you see,
        its ID or file name
~/libyui-port/yast move-app "Clean pacman cache" --reset         # back to its own
        section
~/libyui-port/yast rename-section Hardware --to "Hardware & Fans"
~/libyui-port/yast rename-section Hardware --reset
~/libyui-port/yast list-layout
```

These work for every program and section, including YaST's own, because nothing installed is changed: your layout is a small file, `~/.local/share/yast-on-arch/layout.ini`, that the Qt Control Center reads when it starts:

```
[Moves]
org.archlinux.yast.PacmanCache.desktop=Hardware

[Names]
Hardware=Hardware & Fans
```

It survives rebuilds. If you remove a section that programs were moved into, they go back to their own sections. Moves and renames show in the Qt Control Center; the text-mode and GTK menus (`yast -r menu`, `yast -r -g menu`) still use the installed layout. Names of programs and sections can't contain commas, `"` or `\`, or start or end with a space: YaST would show them differently.

The manual way: drop a file into the repository

If you build from your own clone of this repository, you can also keep entries in it:

- a program: a `.desktop` file in `arch/applications/` (name it `org.archlinux.yast.<Something>.desktop`)
- a section: a `.desktop` file in `arch/applications/groups/`

then run `./bootstrap.sh` again. The build copies them as they are. `@YAST_ROOT@` in an `Exec=` line is replaced with the install folder (`~/libyui-port`), so `Exec=@YAST_ROOT@/yast -r -q my_module` works for any user; plain paths are left alone.

A program entry:

```
[Desktop Entry]
Type=Application
Categories=Settings;X-SuSE-YaST;
X-SuSE-YaST-Group=MyTools
X-SuSE-YaST-SortKey=50
X-SuSE-YaST-RootOnly=false
Icon=utilities-terminal
Exec=konsole --hold -e htop
Name=Process Monitor
GenericName=Process Monitor
Comment=See what is using the CPU
```

A section entry (`X-SuSE-YaST-Group` is the ID that program entries refer to; `X-SuSE-YaST-SortKey` orders the sidebar):

```
[Desktop Entry]
Type=Application
Categories=Settings;Qt;X-SuSE-YaST;
X-SuSE-YaST-Group=MyTools
X-SuSE-YaST-SortKey=90
OnlyShowIn=X-SuSE-YaST;
Icon=applications-utilities
Name=My Tools
GenericName=My Tools
Exec=true
```

Rules that matter: an entry **without** `X-SuSE-YaST-Call` is started by running its `Exec=` line as your user (`sh -c`); an entry **with** `X-SuSE-YaST-Call=<module>` is a YaST module and is started as root through `yast -r -q <module>`. A program whose `X-SuSE-YaST-Group` names no existing section does not appear. `Terminal=true` is ignored by the Control Center; wrap the command in a terminal instead (`konsole --hold -e …`).

---

6. Check that it works

1. `~/libyui-port/yast` prints `YaST on Arch (prefix: ...)` and a long list of clients.
2. Something read-only, as your user: `~/libyui-port/yui-demo HelloWorld` shows a blue ncurses window.
3. YaST's own data layer reading your system (prints your hostname, changes nothing):

```
source ~/libyui-port/yast-env.sh
ruby -e 'require "yast"; include Yast; p SCR.Read(Path.new(".target.string"), "/etc/
        hostname")'
```

4. Hardware agent self-test (read-only). Run it in the same shell, after `source ~/libyui-port/yast-env.sh` from step 3 (it needs YaST's Perl modules): `~/libyui-port/prefix/lib/YaST2/servers_non_y2/ag_probe --selftest`
5. The Control Center opens: `~/libyui-port/yast control-center`.

Logs:

who	where
modules run as root	`/var/log/YaST2/y2log`
modules run as your user	`~/.y2log`
anything started without a terminal (menu entry, Control Center clicks)	`$XDG_RUNTIME_DIR/yast-on-arch.log`

---

7. Safety: read this before using modules as root

YaST is not a viewer. It edits real configuration files and starts and stops real services.

- **There is no dry-run mode.** Pressing **OK**, **Next**, **Finish** or **Accept** writes. Use **Cancel** / **Abort** if you only wanted to look.
- **Some modules write as soon as you press OK, even if you changed nothing.** On this port these four are known to do that:
  - **Boot Loader**: guarded, see below.
  - **Samba Client**: can rewrite `smb.conf`, `nsswitch.conf` and PAM/winbind settings.
  - **Security**: writes `/etc/sysctl.d/70-yast.conf` and edits `/etc/login.defs`. The port shows your live kernel values so it doesn't write wrong defaults, but check every page before OK.
  - **NTP Client**: can enable `chronyd` and so turn off `systemd-timesyncd`.
- **Boot Loader writing is switched off.** Patch 12 refuses to write the GRUB configuration unless the environment variable `YAST_ALLOW_BOOTLOADER_WRITE=1` is set. You get a popup saying "Nothing was changed". To allow it deliberately, set it on the command line: `YAST_ALLOW_BOOTLOADER_WRITE=1 ~/libyui-port/yast -r -q bootloader`. The launcher passes the variable on to root only when you set it to exactly `1`, and never sets it itself. **Back up** `/etc/default/grub` **and** `/boot/grub/grub.cfg` **first.**
- **The Partitioner (**`disk` **/** `storage`**) can destroy data** if you apply a plan. Treat it like any partition editor.
- **Never run a module from a script with input closed or piped** (`yast -r security </dev/null`, `echo | yast …`, `script … </dev/null`). In text mode, the end of input counts as pressing the **default button**. The module then walks itself through every dialog to OK/Finish and **writes**. If a script must start a module, keep its input open and empty (for example `sleep 600 | yast …`) and watch it.
- **Packages are never installed behind your back.** When a module needs a package, the `Pkg` stand-in shows the exact `pacman -S …` command for you to run, and reports the install as not done.
- **Before trying a new module, keep copies** of the files it owns, and compare afterwards:

```
sudo cp -a /etc/samba/smb.conf /etc/default/grub /etc/login.defs ~/yast-backup/ 2>/
    dev/null
sudo find /etc -newer ~/yast-backup -type f     # after closing the module: what
    changed?
```

---

8. Reference: commands, flags and variables

`~/libyui-port/yast`

```
yast [-r] [-q|-g] <module> [module args...]
yast control-center | cc [Control Center args...]
yast add-app | remove-app | add-section | remove-section | list-apps | list-sections |
     move-app | rename-section | list-layout | sync-apps
(section 5)
yast --version
yast                        (list modules)
```

flag	meaning
`-r`	Run as root: `sudo` from a terminal; without a terminal `sudo -n`, else `pkexec`, else a terminal window, else a visible error (section 4). The launcher passes every path YaST needs to root explicitly (`sudo` / `pkexec` would otherwise drop them). For Qt/GTK it also allows root on your display (`xhost +SI:localuser:root`), finds your X authority cookie, and forces XWayland (`QT_QPA_PLATFORM=xcb`, `GDK_BACKEND=x11`).
`-q`	Qt front-end.
`-g`	GTK front-end (Adwaita, dark unless `YAST_GTK_THEME` says otherwise).
`control-center`, `cc`	Start the Qt Control Center as your user.

Environment variables

variable	used by	meaning
`YAST_ARCH_ROOT`	`bootstrap.sh`, `yast-env.sh`	Install location. Default `~/libyui-port`.
`YAST_DEBUG`	`yast`	Any value: with `-r`, print the escalation method that would be used and the exact `sudo env …` command, and exit without running it.
`YAST_ESCALATE`	`yast -r`	Force how root is obtained: `sudo`, `pkexec` or `terminal`. Default: chosen automatically (section 4).
`TERMINAL`	`yast -r`	Preferred terminal emulator when the launcher has to open one.
`YAST_UPSTREAM_UNPINNED`	`bootstrap.sh`	Any value: clone upstream's current branches instead of the pinned commits (section 9).
`YAST_IGNORE_PATCH_FAILURES`	`bootstrap.sh`	Any value: warn and continue when a patch doesn't apply, instead of stopping.
`YAST_NO_MENU_ENTRIES`	`bootstrap.sh`	Any value: don't install the menu entries.
`YAST_GTK_THEME`	`yast -g`	GTK theme, default `Adwaita:dark` (try `Adwaita` for light).

variable	used by	meaning
`YAST_ALLOW_BOOTLOADER_WRITE`	Boot Loader	`1` allows writing GRUB (section 7). Passed through `-r` only when set to exactly `1`.
`YAST_LOG_DIRECT`	`yast`	Internal: set by the Control Center, which already redirects output to the log.
`YAST_LAUNCHER`, `YAST_Y2BASE`, `YAST_UI`, `YAST_DESKTOP_USER`, `YAST_DESKTOP_RUNTIME_DIR`	internal	Set by the launcher so the Control Center and menu can start modules, and hand desktop apps back to your session.
`YUI_PREFERRED_UI`	`yui-demo`	`ncurses` or `qt`.
`XDG_DATA_HOME`	`yast add-app`	Your entries go to `$XDG_DATA_HOME/yast-on-arch/applications` (default `~/.local/share/…`).

`yast-env.sh`

`source ~/libyui-port/yast-env.sh` gives a shell the full YaST environment. Use it for developing, for running Ruby against YaST, and for `ycpc` (the YCP compiler). It sets `PATH`, `LD_LIBRARY_PATH`, `PKG_CONFIG_PATH`, `RUBYLIB` (including the user-gem folders, because YaST's embedded Ruby doesn't activate rubygems), `GEM_PATH`, `Y2DIR`, `Y2BASE_PLUGINDIR`, `PERL5LIB` (Perl modules such as Users and Samba), `LIBSTORAGE_CONFDIR`, `AUGEAS_LENS_LIB`, and `CC=gcc CXX=g++`. It works when sourced from bash. From zsh it finds itself too; otherwise it falls back to `$YAST_ARCH_ROOT`.

`bootstrap.sh`

form	does
`./bootstrap.sh`	Full build and install (safe to re-run: existing clones are reused, patches are skipped if already applied).
`./bootstrap.sh --deps`	Print the pacman and gem lists and exit.

Files outside the prefix

file	when	why
`/etc/systemd/system/ypbind.service.d/50-yast-arch-args.conf`	only if `ypbind-mt` is installed	Arch's `ypbind` unit takes its options on the command line. This drop-in reads them from `/etc/conf.d/ypbind`, which is where the NIS Client module saves them.
`~/.local/share/applications/yast-*.desktop`	unless `YAST_NO_MENU_ENTRIES` is set	menu entries
`~/.local/share/yast-on-arch/applications/`	only when you use `yast add-app`	your own Control Center entries (section 5)
`~/.local/share/yast-on-arch/layout.ini`	only when you move a program or rename a section	your layout (section 5)
`~/.config/yast-on-arch/section-colors.ini`	when you change *Section colours* or add a genre	colour mode, colours and aliases (section 4)

file	when	why
`/var/log/YaST2/`	you create it	root log folder
user gems in `~/.local/share/gem/`	always	YaST's Ruby dependencies

---

9. Updating

```
cd yast-on-arch
git pull
./bootstrap.sh
```

`bootstrap.sh` reuses the upstream clones already in `~/libyui-port`. To pick up **upstream** YaST/libyui changes too, delete the clone(s) you want refreshed (for example `rm -rf ~/libyui-port/yast-users`) and run it again.

> The script clones each upstream repository at a **pinned commit**: the one the patches were made and tested against (the `PIN` table at the top of `bootstrap.sh`). Existing clones are reused as they are; the build warns when one isn't at its pinned commit. If a patch doesn't apply, the build stops and names it. `YAST_UPSTREAM_UNPINNED=1` clones upstream's current code instead (patches may then fail), and `YAST_IGNORE_PATCH_FAILURES=1` continues past a failed patch (that module may then misbehave).

After an Arch **Ruby** or **Qt** upgrade, re-run `bootstrap.sh`: the bindings are compiled against them.

10. Uninstall

```
rm -rf ~/libyui-port                                    # the whole port
sudo rm -f /etc/systemd/system/ypbind.service.d/50-yast-arch-args.conf && sudo systemctl
        daemon-reload
gem uninstall --user-
        install fast_gettext cheetah simpleidn abstract_method yast-rake cfa cfa_grub2
        ruby-dbus prime
sudo rm -rf /var/log/YaST2                              # logs
xhost -SI:localuser:root                                # if a root GUI was opened this
        session
rm -f ~/.local/share/applications/yast-*.desktop        # the menu entries
rm -rf ~/.local/share/yast-on-arch                      # programs you added with yast
        add-app
```

Packages installed by `bootstrap.sh` are ordinary Arch packages. Remove the ones you don't need with `pacman -Rs`, carefully: other software may use them too.

Configuration that YaST modules wrote (users, smb.conf, sysctl, enabled services...) stays. That is real system configuration and it doesn't depend on YaST.

---

11. Troubleshooting

symptom	cause and fix
`yast-env.sh: prefix not found`	The build didn't finish, or you used `YAST_ARCH_ROOT` for the build but not afterwards.
`FAIL yast-<module>` during the build	A module's `rake install` failed. Re-run; if it persists, upstream changed. Check `cd ~/libyui-port/yast-<module> && rake install DESTDIR=../prefix/destdir`.
`component cannot import namespace '...'`	A Ruby binding or plugin isn't where YaST looks. Re-run `bootstrap.sh`. For `Pkg`, the Arch stand-in is missing from `prefix/destdir/usr/share/YaST2/modules/`.
`undefined method '#<Symbol:0x...>'`	yast-ruby-bindings was built without patch 03 (Ruby 3.4+). Delete `~/libyui-port/yast-ruby-bindings` and re-run.
`Can't locate ycp.pm` / `YaST/YCP.pm`	`PERL5LIB` not set. Use the launcher, or source `yast-env.sh`.
`fast_gettext` errors	Version 3.x got in. `gem install --user-install fast_gettext -v '<3.0'`.
Clicking a module in the Control Center does nothing	No password dialog appeared: check that a polkit agent is running, or set `YAST_ESCALATE=terminal`. See section 4 and `$XDG_RUNTIME_DIR/yast-on-arch.log`.
`ERROR: patch … does not apply` during the build	The clone isn't at its pinned commit (an old clone, or `YAST_UPSTREAM_UNPINNED`). Delete that clone in `~/libyui-port` and re-run.
Root Qt/GTK window dies instantly, `cannot open display`	`xorg-xhost` missing (the launcher prints a warning), no XWayland, or no X cookie found. Install `xorg-xhost`; run from your desktop session.
GTK front-end aborts (`ensure_surface_for_gicon`, `Loader process exited early`)	Your icon theme lives in `~/.local/share/icons`, which GTK's sandboxed SVG loader can't read. The launcher works around it; make sure you start with `-g` through the launcher, not `y2base` directly.
Qt module as root looks light / unthemed	Your `~/.config/kdeglobals` is missing (non-KDE desktop). Harmless.
`services-manager` takes minutes to open	Normal: two `systemctl` calls per unit, and a typical Arch box has around 1,000 units.
`Target display-manager.target / syslog.target not found`, Nokogiri libxml warning, `already initialized constant TMP_RUBY_PREFIX`	Harmless SUSE-isms and warnings.
A module asks to install an openSUSE package name	The `Pkg` stand-in maps common names to Arch's and shows the `pacman` command; run it yourself, then reopen the module.
Boot Loader: "Writing the boot loader configuration is disabled on this port"	Intended (section 7).

symptom	cause and fix
A program added with `yast add-app` doesn't show up	Restart the Control Center. Check `yast list-apps`; its section must exist (`yast list-sections`). After a rebuild, `yast sync-apps`.
`yast add-app`: "program '...' not found"	The first word of `--exec` isn't on your `PATH`. Use the full path, or `--force` if you will install it later.
A hand-made entry in `arch/applications/` doesn't appear	Its `X-SuSE-YaST-Group` must match a section's ID, and the file must end in `.desktop`. Re-run `bootstrap.sh`.
Language module complains on a fresh install	Fixed by patch 16 (`LANG=C.UTF-8` is treated as "no localization"). If you see it, re-run `bootstrap.sh`.

12. What was changed for Arch (the patches)

patch	component	change
—	libyui-ncurses	ncursesw header shim (build-time only)
01	yast-devtools	find DocBook XSL at Arch's path
02	yast-devtools	don't install RPM macros
03	yast-ruby-bindings	**Ruby ″ 3.4: full VM initialisation** (section 13)
04	(example)	how an SCR agent is pointed at a new path: a `sed`, not code
05	yast2 `menu.rb`	start modules through `y2base` (no `/sbin/yast2`); hand desktop-only entries back to the user session
06	yast2 `Directory.rb`	data/desktop paths follow the prefix
07	yast2-control-center	launch via the `yast` launcher, theme dir in the prefix, RootOnly modules visible, never launch an empty client
08	libstorage-ng	honour `--prefix`
09	yast-bootloader	`grub-*` tools and `/boot/grub` (not `grub2-*`, `/boot/grub2`); no shim-install
10	yast2 `Arch.rb`	architecture from `uname -m` when no hardware probe exists
11	yast2 PackageSystem	ask the pacman-backed `Pkg`, not rpm
12	yast-bootloader	GRUB write guard (`YAST_ALLOW_BOOTLOADER_WRITE`)
13	yast-users	load the X500::DN Perl module only for LDAP
14	yast-security	yescrypt known; show live `/proc/sys` values when no sysctl file sets a key
15	yast2 ShadowConfig	edit `/etc/login.defs` itself (Arch's shadow has no `login.defs.d`)
16	yast-country	`LANG=C.UTF-8` is "not localized", not an error
17	yast-country	keep the console default when `/etc/sysconfig/console` is absent
18	yast-tftp-server	Arch `/etc/conf.d` file and unit names
19	yast-nis-client	ypbind settings in `/etc/conf.d/ypbind` (+ the systemd drop-in)
20	yast2-control-center	search no longer freezes with thousands of modules
21	yast2-control-center	group sidebar narrows to groups with matching modules while searching
22	yast2-control-center	section colours (Off / Headers / Whole block; a colour per "More - System" category; `[Colors]` and `[Aliases]` in `section-colors.ini`); "Add or change…" window; moved programs and renamed sections from `layout.ini`

Plus the Arch stand-ins in `arch/`: `Pkg` (pacman-backed queries, no transactions), no-op `InstURL` / `Packages` / `SLPAPI` / `PackageSlideShow`, `cfa/grub2` copies for `/boot/grub`, a Perl `.probe` agent that reads `/sys`, `/proc` and DMI instead of hwinfo (`docs/probe-agent.md`), and an Arch `login_defs` augeas lens.

---

13. FAQ

Is this the real YaST? Yes. The upstream openSUSE source code, built for Arch with small patches. No YaST code is copied into this repository.

Why not Cockpit / KDE System Settings? Different tools. This project exists for people who want YaST specifically. (KDE's own modules are reachable from inside YaST through the KDE Plasma Settings module.)

Does it change my system just by being installed? It installs build dependencies with pacman and user gems, its menu entries in `~/.local/share/applications`, and nothing else outside `~/libyui-port` (except the NIS drop-in when `ypbind-mt` is present). Modules change your system only when you use them.

Can I manage a remote server? Install it there and use text mode over SSH: `ssh -t server ~/libyui-port/yast -r menu`.

Why does YaST insist on root even though I'm in `wheel`? It checks for uid 0 literally. Use `-r`.

What is the "Ruby 3.4 fix" everybody else might want? yast-ruby-bindings starts its embedded Ruby with a hand-copied subset of Ruby's start-up code. On Ruby 3.4 that leaves out Ruby's prelude, and loading rubygems fails with nonsense like `undefined method '#<Symbol:0x…>'`. Patch 03 calls the real `ruby_options()` with an empty `-e` script instead. That initialises Ruby completely and runs nothing. Any distribution on Ruby 3.4+ needs this.

Will Software Management ever work? Not as YaST's own package manager: that would mean writing a libalpm backend for libzypp's interface. Octopi fills the gap.

---

14. Glossary

- **YaST** — SUSE's system configuration tool. **Module / client** — one tool inside it (Users, Services Manager…); "client" is YaST's internal word.
- **Control Center** — the window listing all modules by group.
- **Section / group** — one entry of the Control Center's sidebar (System, Network…). Each is a small `.desktop` file in `groups/`; its `X-SuSE-YaST-Group` value is the section ID.
- `.desktop` **file** — the standard Linux text file describing a menu entry (name, icon, command). YaST uses the same format with extra `X-SuSE-YaST-…` keys.
- **libyui** — the widget library behind YaST's three looks (Qt, GTK, ncurses).
- **ncurses / TUI** — text-mode user interface in a terminal.
- **y2base** — the program that actually runs a YaST module.
- **YCP** — YaST's original scripting language; modern modules are Ruby, some are Perl.
- **SCR** — YaST's layer for reading and writing system configuration. **SCR agent** — a small description or program that knows one file or data source (e.g. `.target.string`, `.probe`).
- **Prefix** — the private install folder (`~/libyui-port/prefix`).
- **destdir** — a staging folder inside the prefix where Ruby parts are installed.
- **Pkg stand-in** — this port's replacement for YaST's libzypp package bindings; it answers queries from pacman and never installs.
- **libzypp / RPM** — openSUSE's package-management library and package format.
- **libstorage-ng** — SUSE's storage library behind the Partitioner.
- **augeas / lens** — a configuration-file editing library, and its per-format parser.
- **KCM** — a KDE configuration module (one page of System Settings).
- **XWayland / xhost** — X11 compatibility on Wayland, and the command that allows another user (root) to draw on your display.
- **sysctl** — kernel settings under `/proc/sys`, persisted in `/etc/sysctl.d/`.
- **NIS / ypbind** — the old Network Information Service for shared user accounts, and its client daemon.

---

15. Credits and licence

- **YaST** — © SUSE LLC and the openSUSE contributors (github.com/yast), GPL-2.0.
- **libyui** — © SUSE LLC and contributors (github.com/libyui), LGPL-2.1/GPL as per each component.
- **libstorage-ng** — © SUSE LLC (github.com/openSUSE/libstorage-ng).
- **Octopi** — Alexandre Albuquerque Arnt and contributors (optional, separate project).
- **yast-on-arch** — build script, patches and Arch stand-ins by Eugene Flanagin. GPL-2.0, matching upstream; the patches are derivative of GPL-2.0 code.

yast-on-arch changes are Copyright (C) 2026 by Eugene Flanagin. Licensed under the GNU General Public License v2.0 (GPL-2.0), like YaST itself.

SUSE, openSUSE and YaST are trademarks of SUSE LLC. Arch Linux is a trademark of the Arch Linux project. KDE and Plasma are trademarks of KDE e.V. This project is not affiliated with, endorsed by, or supported by SUSE, openSUSE or Arch Linux.